Module 04 - Report

Stefano Vitale

1 Distribution Strategy

The MPI implementation partitions the work across N_{ranks} ranks, where N_{ranks} must be a power of two. The distributed pipeline is formalized using a Bulk Synchronous Parallel (BSP) model, as illustrated in Figure 1, alternating between computation supersteps and global synchronization barriers. To simulate a real-world scenario with a single storage node, raw data is generated sequentially on Rank 0 and distributed via `MPI_Scatterv` (with the final rank absorbing any remainder). Rank 0 then frees its global buffers. From this point forward, all ranks execute identical, symmetric code paths.

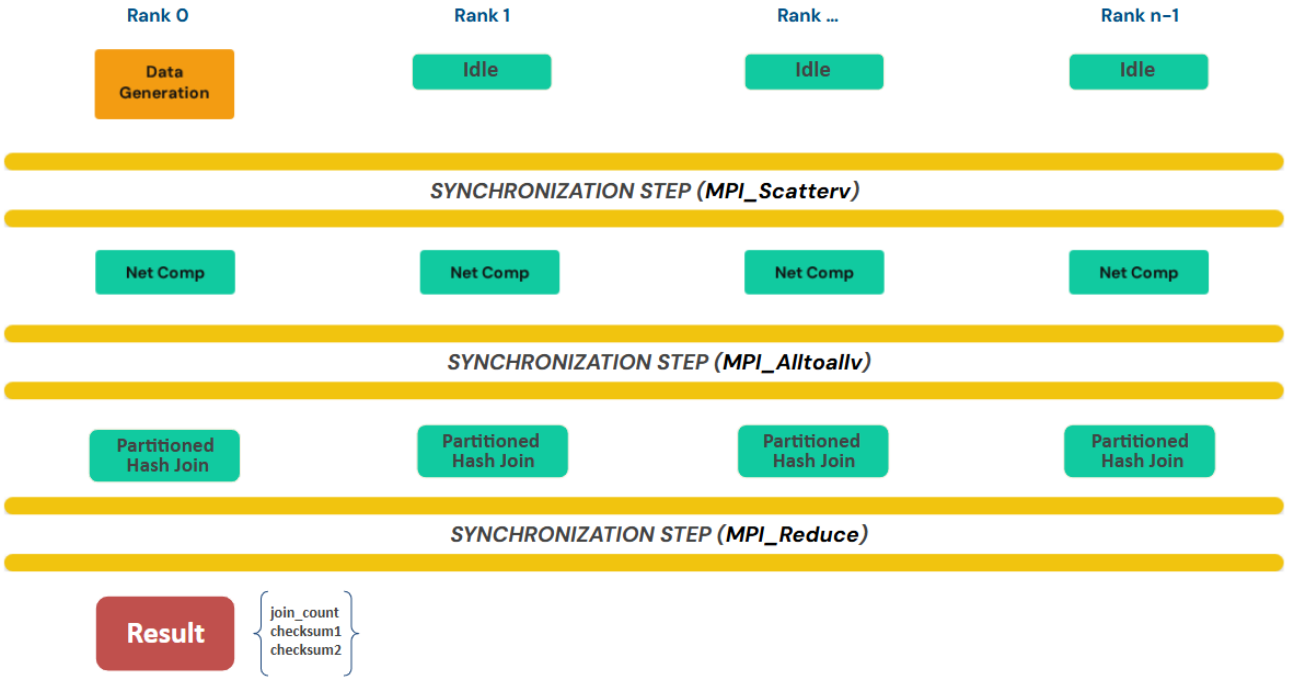


Figure 1: Bulk Synchronous Parallel (BSP) model representation of the distributed hash join pipeline.

1.1 Two-Level Partitioning: Macro-Partitioning and Micro-Partitioning

The same three-step pattern HISTOGRAM–PREFIX-SUM–SCATTER (hereafter *HPS*) is applied at two distinct levels.

Level 1 : Macro-Partitioning (Network routing). Before any communication, each rank applies HPS locally to its chunk. The destination rank for a record with key k is:

$$dest\_rank = \texttt{hash\_murmur3\_32}(k, \log_2(N_{\text{ranks}}))$$

1. **Histogram**: count how many local records map to each destination rank.
2. **Prefix sum**: compute send displacements from the histogram.
3. **Scatter**: reorder local records into N_{ranks} contiguous groups.

A derived MPI datatype (`MPI_Type_create_struct`) is defined for the `Record` structure to ensure portability and correct memory alignment. Send counts are exchanged via `MPI_Alltoall`; records are exchanged via `MPI_Alltoallv`. After this phase, rank r owns all records from both R and S whose key hashes to r.

Level 2 : Micro-Partitioning (local partitioned hash join). Each rank calls `partition_relation` on its received buffers, applying HPS a second time. The local partition for key k is:

$$local\_partition\_id = \texttt{hash\_murmur3\_32}(k, \log_2(P_{\text{per\_rank}}))$$

This produces $P_{\text{per\_rank}}$ contiguous sub-partitions per rank, enabling the standard build–probe loop over each partition independently. The sub-partition function is identical to the sequential baseline (`_seq.cpp`), so the local join logic is reused without modification.

Hash function and correctness. Both partitioning levels utilize the Murmur3 `fmix32` finalizer. Its deterministic nature guarantees that matching keys from R and S always co-locate on the same rank and local partition, satisfying the necessary condition for a partitioned hash join. Furthermore, as extensively analyzed in Module 1, its avalanche properties are crucial to mitigate rank imbalance, especially under skewed workload distributions.

Timing scope and barrier synchronization. The centralized setup phase (generation and `MPI_Scatterv`) is excluded from the measured execution time, decoupling the inherent single-node initialization overhead from the core algorithmic performance. An `MPI_Barrier` is enforced immediately before the main timer to establish a synchronous baseline. This barrier ensures all ranks enter the measured region simultaneously, so the reported time reflects true parallel execution rather than inter-rank skew.

1.2 Analytical Cost Model and Scalability Limits

To evaluate the empirical performance of the parallel join, a theoretical cost model is established based on the volume of data transferred across the pipeline. Let N be the number of records per relation ($N_R = N_S = N$), and R be the number of MPI ranks. Under a perfectly uniform key distribution, the system behaves as follows:

- **Computation and Work per Rank:** After the initial centralized data ingestion, each rank receives a raw slice of size N/R for relation R and N/R for relation S. The local join cost is therefore $O(N/R)$. Under perfect strong scaling, the compute time decreases linearly as the rank count R increases.
- **Communication (Network Redistribution):** The `MPI_Alltoallv` phase represents the primary scalability bottleneck. Each rank applies the macro-partitioning hash to its chunk ($2N/R$ total records) and distributes them evenly across all $R-1$ ranks. The payload volume exchanged between any specific pair of ranks is:

  $$\text{Volume}_{\text{per-pair}} = \frac{2N/R}{R} = \frac{2N}{R^2} \quad \text{records}$$

  For instance, in a strong scaling scenario with $N = 50M$ and $R = 64$, the theoretical payload transmitted between two distinct ranks is exactly $100M/64^2 \approx 24,414$ records. Given an 8-byte record structure, this translates to ≈ 195 KB per network link. However, while the data volume per link decreases quadratically, each rank must maintain $R-1$ concurrent connections. Consequently, the total outgoing volume per rank remains `O(N/R)`; the scalability challenge of `MPI_Alltoallv` arises not from total volume, but from the number of concurrent connections and their sensitivity to simultaneous large payloads.
- **Global Reduction:** After the local join, each rank transmits only a scalar aggregate (join count and checksums) via `MPI_Reduce`.

2 Performance and Scalability Analysis

All experiments use $N_R = N_S = N$ records with uniform key distribution. Two task-density configurations are evaluated simultaneously: 8 and 16 tasks per node (hereafter called 8T/N and 16T/N). All reported phase times are the *maximum* across all ranks, obtained via `MPI_Reduce` with `MPI_MAX`. This ensures that the measured time for each phase corresponds to the slowest rank, reflecting the true execution time.

2.1 Data Size Scaling and Overall Speedup

Table 1: Data size scaling (fixed: 4 nodes, 32 MPI tasks).

N	Seq (s)	MPI (s)	Speedup
1M	0.060	0.253	0.23×
25M	2.006	0.535	3.74×
50M	4.032	0.618	6.52×
100M	8.059	0.821	9.81×
200M	16.276	1.308	12.43×

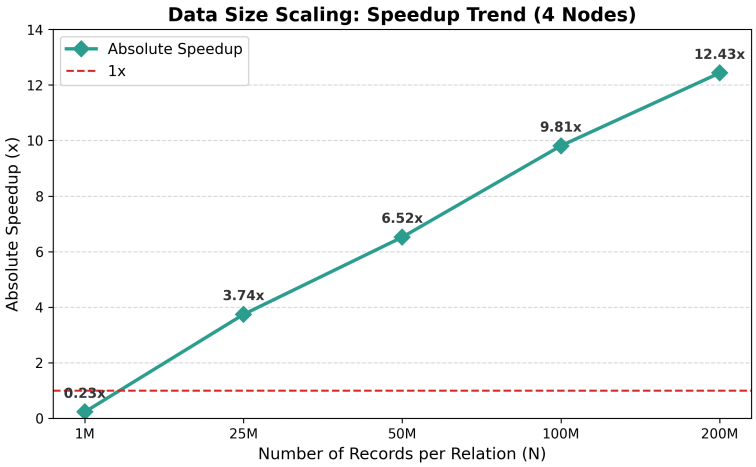


Figure 2: Absolute speedup vs. dataset size.

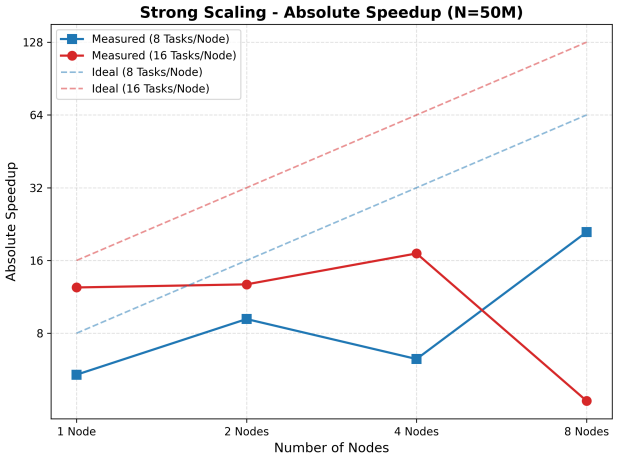
Table 1 and Figure 2 show that system performance is dominated by the ratio of computation to communication overhead. At $N = 1$M, network startup and synchronization costs outweigh the minimal computational workload, yielding a 0.23× speedup. As N grows, the computational workload amortizes these fixed overheads: the sequential time scales roughly linearly with N, while the MPI time grows more slowly.

2.2 Strong Scaling Analysis

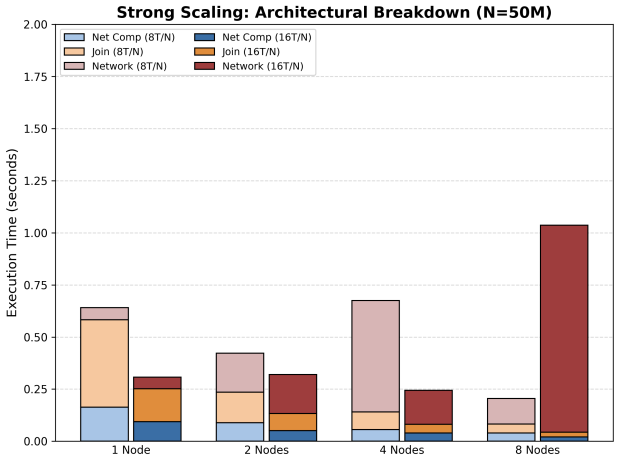
Strong scaling experiments fix $N = 50$M, from 1 to 8 nodes. Table 2 reports the measured time, absolute speedup, parallel efficiency, and the breakdown between compute and communication.

N (T)	MPI (s)	Speedup	Eff.	Net Comp (s)	Join (s)	Comm (s)	%
Configuration: 8 Tasks per Node							
1 (8)	0.639	5.39×	67.4%	0.164	0.419	0.058	9%
2 (16)	0.415	9.16×	57.2%	0.089	0.146	0.187	45%
4 (32)	0.647	6.26×	19.5%	0.056	0.085	0.535	83%
8 (64)	0.201	20.97×	32.8%	0.039	0.043	0.123	61%
Configuration: 16 Tasks per Node							
1 (16)	0.305	12.37×	77.3%	0.094	0.159	0.055	18%
2 (32)	0.317	12.74×	39.8%	0.051	0.082	0.187	59%
4 (64)	0.245	17.11×	26.7%	0.040	0.042	0.163	66%
8 (128)	1.036	4.20×	3.3%	0.021	0.022	0.993	96%

Table 2: N (T): Nodes (Total MPI Ranks). **Note**: since component times report the maximum across all ranks, their sum may slightly exceed the total execution time, as the bottlenecks may occur on different ranks.



(a) Absolute speedup.



(b) Phase breakdown: computation vs. communication.

Figure 3: Strong scaling: absolute speedup and architectural breakdown for $N = 50$M.

Linear scaling of the compute phases. Across all configurations, the execution times of both local components (Net Comp and Join) exhibit ideal strong scaling, halving linearly as the rank count doubles. This confirms that both the macro-partitioning preparation and the local hash-join scale perfectly with the distributed data volume, validating the analytical cost model.

Non-monotonic network behavior in the 8T/N configuration. The 8T/N curve shows a clear regression at 4 nodes: time increases from 0.415 s (2 nodes) to 0.647 s (4 nodes), and speedup drops from 9.16× to 6.26×. The phase breakdown isolates the cause: communication time jumps from 0.187 s to 0.535 s. Finally, communication time decreases to 0.123 s at 8 nodes, yielding the best overall result of 20.97×.

This non-monotonic pattern highlights the trade-off of the `MPI_Alltoallv` collective: message size versus connection count.

- **At 4 nodes ($R = 32$)**, the per-pair message size is $\frac{2N}{R^2} = \frac{100\text{M}}{32^2} \approx 97{,}656$ records × 8 B ≈ 780 KB. Simultaneously injecting these large payloads across 32 concurrent connections is the most plausible cause of the 0.535 s communication spike; direct network profiling was not performed, so this remains an architectural inference consistent with the observed timing.
- **At 8 nodes ($R = 64$)**, the per-pair message shrinks to $\frac{100\text{M}}{64^2} \approx 24{,}414$ records × 8 B ≈ 195 KB (4× reduction). This appears sufficient to keep concurrent traffic within buffer capacity, allowing communication to recover to 0.123 s despite the doubled connection count.

Degradation in the 16T/N Configuration at 8 Nodes. At 128 total ranks, communication reaches 0.993 s (96% of total time) with extreme run-to-run variance, collapsing efficiency to 3.3% and rendering this configuration effectively unusable for this problem size.

Root Cause Spawning 16 MPI tasks utilizes all physical cores for computation but creates a severe inter-node communication bottleneck. At $R = 128$, 16 local tasks simultaneously inject traffic into 127 peer connections. All 16 ranks on the same node share a single physical NIC, so the available inter-node bandwidth is divided among 16 simultaneous senders, leaving each rank with a fraction of the total link capacity.

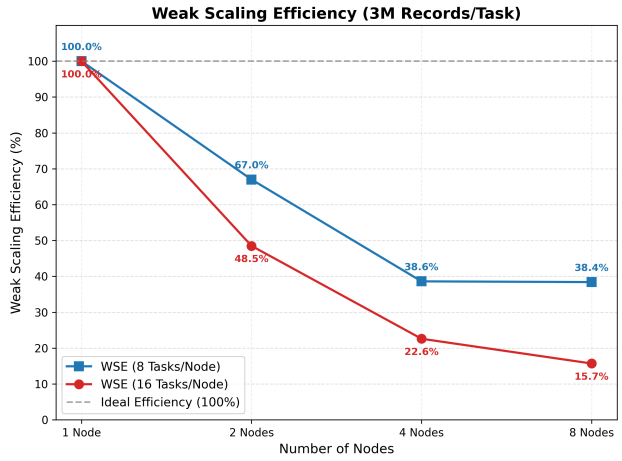
Possible Architectural Solution MPI-Based To resolve this limitation without leaving physical cores idle, the implementation could shift to a *Hierarchical Communication* model. By splitting `MPI_COMM_WORLD` into node-local communicators, ranks residing on the same node could first aggregate their outgoing payloads. Subsequently, the inter-node exchange would be restricted to a single representative rank per node, drastically shrinking the global communication matrix from 128×128 down to just 8×8. Finally, the received payload would be scattered intra-node. Alternatively, a Hybrid shared-memory approach can resolve this contention, as explored in Section 4.

2.3 Weak Scaling Analysis

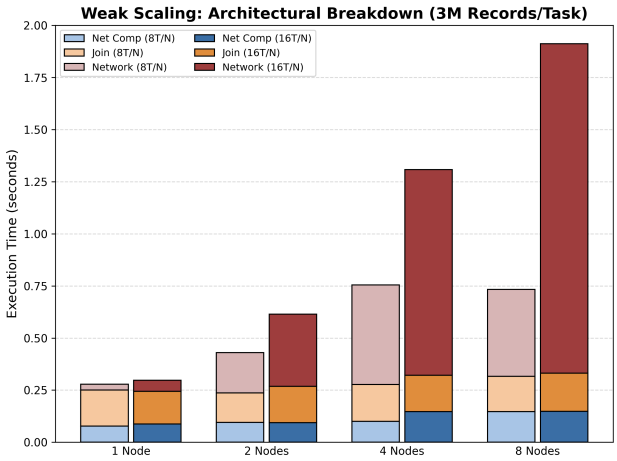
Weak scaling experiments fix 3M records per rank and scale the cluster from 1 to 8 nodes, so the total problem size grows proportionally with the rank count. Table 3 reports time, weak scaling efficiency (WSE), and the phase breakdown.

N (T)	Size	MPI (s)	WSE	Net Comp (s)	Join (s)	Comm (s)
Configuration: 8 Tasks per Node						
1 (8)	24M	0.278	1.000	0.078	0.173	0.028
2 (16)	48M	0.414	0.670	0.095	0.142	0.193
4 (32)	96M	0.719	0.386	0.101	0.176	0.478
8 (64)	192M	0.722	0.384	0.147	0.170	0.416
Configuration: 16 Tasks per Node						
1 (16)	48M	0.295	1.000	0.088	0.156	0.053
2 (32)	96M	0.608	0.485	0.095	0.173	0.346
4 (64)	192M	1.307	0.226	0.147	0.174	0.987
8 (128)	384M	1.885	0.157	0.148	0.184	1.579

Table 3: N (T): Nodes (Total MPI Ranks)



(a) Weak scaling efficiency (WSE).



(b) Phase breakdown: compute vs. communication.

Figure 4: Weak scaling: efficiency and architectural breakdown at 3M records/rank.

Constant local computation. As confirmed in Figure 4b, the execution times of both Net Comp and Join remain nearly constant across all node counts in both configurations. This confirms that each rank computes its fixed 3M-record chunk without degradation, regardless of the cluster size.

WSE is driven predominantly by communication. The efficiency drop is dominated by network communication.

For **8T/N**, communication grows from 0.028 s at 1 node to 0.478 s at 4 nodes, accounting for 67% of total time. Between 4 and 8 nodes the total time remains essentially flat (0.719 s $\rightarrow$ 0.722 s), holding WSE at $\approx$ 38.4%. This is consistent with the cost model: in weak scaling, the per-pair message size is $6\text{M}/R$, so doubling the rank count from 32 to 64 halves it (from $\approx$1.5 MB to $\approx$750 KB). This reduction compensates for the cost of managing additional connections, preventing further growth in communication time.

Conversely, **the 16T/N** configuration exhibits continuous degradation. Communication grows monotonically from 0.053 s to 1.579 s, reaching 83.8% of the total time at 8 nodes, dropping the WSE to 15.7%. The contrast between the two configurations at 8 nodes (0.722 s vs. 1.885 s) confirms that intra-node network contention is the primary limiting factor for MPI scaling.

3 Distributed vs. Shared-Memory Architecture

This section compares the distributed MPI architecture with the shared-memory OpenMP implementations developed in Module 3. To guarantee a fair architectural evaluation, the comparison focuses on a fixed computational baseline: executing the $N = 50M$ uniform dataset utilizing exactly 16 processing units across different models.

The 16-Core Architectural Comparison. Table 4 contrasts the three paradigms. The shared-memory OpenMP baseline completes the workload in 0.262 s. When transitioning to a MPI model confined to a single node (1 Node $\times$ 16 Tasks), the execution time rises to 0.305 s. This 16% penalty is strictly local: MPI ranks cannot share data directly and must explicitly redistribute records via memory copies, adding overhead absent in the OpenMP model.

When the same 16 MPI ranks are distributed across two physical machines (2 Nodes $\times$ 8 Tasks), the execution time jumps to 0.415 s. In this topology, the intra-node memory bus congestion is relieved, but the execution must cross the physical network link, introducing inter-node communication latency.

Architectural Trade-off. Shared-memory OpenMP maximizes intra-node efficiency but is bounded by single-node limits. Conversely, pure MPI increases the scaling possibilities but introduces communication overhead

Execution Model	Topology (16 Cores)	Time (s)	Primary Bottleneck
OpenMP Loop (Module 3)	1 Node × 16 Threads	0.262	Local Memory Bus
MPI (Module 4)	1 Node × 16 Ranks	0.305	Intra-node Message Passing
MPI (Module 4)	2 Nodes × 8 Ranks	0.415	Network

Table 4: Performance comparison at a fixed 16-core ($N = 50M$).

at high task densities. Optimizing the pure MPI implementation requires lowering the rank density to prevent network congestion, which inevitably leaves physical cores idle.

4 Hybrid MPI+OpenMP Architecture

As demonstrated in the strong scaling analysis, pushing the MPI approach to full intra-node saturation (16 Tasks/Node) yields poor results. The 128 isolated ranks force the `MPI_Alltoallv` collective to manage a dense 128×128 connection matrix, causing network contention to completely dominate the execution (96% of total time).

This section explores the architectural solution to this bottleneck: replacing 16 isolated MPI processes with a single MPI rank distributing the workload across 16 OpenMP threads.

Execution Model	Topology (8 Nodes)	Total Time	Speedup	Net Comp	Join	Comm
MPI	128 Ranks × 1 Thread	1.036 s	1.00×	0.021 s	0.022 s	0.993 s
MPI+OMP	8 Ranks × 16 Threads	**0.308 s**	**3.36**×	0.147 s	0.047 s	0.116 s

Table 5: Performance impact of the hybrid MPI+OpenMP model vs MPI at 8 nodes ($N = 50M$).

Results Discussion. By shifting from a pure MPI implementation to a hybrid model, the network topology was reduced from a dense 128×128 graph to a sparse 8×8 graph. This completely resolved the network contention, reducing the communication time of `MPI_Alltoallv` from 0.993 s to just 0.116 s (an 88.3% reduction).

In the hybrid model, the macro-partitioning phase runs sequentially on the single MPI rank per node. Each rank therefore processes 16× more records than its pure MPI counterpart. While this increases the data volume per rank by 16×, the measured cost (0.147 s) scales sub-linearly, resulting in a 7× increase compared to the pure MPI baseline (0.021 s). This behavior demonstrates that the single-rank topology successfully eliminates intra-node memory bandwidth contention: in the pure MPI case, 16 ranks on the same node compete for the memory bus simultaneously during HPS, heavily inflating the per-record cost.

The local join phase (`partitioned_hash_join_parallel` developed in Module 3) runs fully parallel across all 16 OMP threads, with independent per-partition workloads requiring no synchronization.

Justification of $P_{local} = 2048$ in the Hybrid Architecture In the hybrid MPI+OpenMP configuration, the number of local micro-partitions was set to $P_{local} = 2048$. This configuration mirrors the shared-memory environment of Module 3. Retaining $P_{local} = 2048$ reuses the optimal data granularity for the L1/L2 caches, ensuring ideal core utilization without redundant parameter tuning.

5 Correctness and Methodology

To ensure a rigorous and fair evaluation, the MPI implementation was extensively validated against the sequential baseline. Tests via a dedicated verification script (`submit_naive.sh`) confirmed that all implementations yield identical outputs (join_count, checksum1, and checksum2) across all uniform and skewed datasets. For small problem sizes, the verification script also triggers a purely sequential nested-loop join ($O(R \times S)$ complexity) to act as reference, guaranteeing the mathematical correctness of the baseline itself.

To evaluate network noise and observe the variance between consecutive executions, all experiments were run 3 times. The reported metrics represent the arithmetic mean of these runs.

Baseline Observations and Algorithmic Fairness The sequential baseline is executed with P = N_ranks × P_local partitions, matching the total partitioning depth of the distributed implementation. This matching ensures that the measured speedup reflects only the distribution overhead, not a difference in partitioning depth; the resulting baseline (4.0s vs 3.2s in Module 3) is intentionally conservative. Run-to-run variance across the cluster is $\leq$ 5–10%.

A Note on Data Skewness While the Murmur3 avalanche effect excellently handles naturally clustered keys, it cannot resolve frequency skew. To establish an absolute worst-case upper bound for the algorithm's performance degradation, Table 6 reports the phase breakdown for an artificial edge case: a dataset where 50% of the keys are identical duplicates.

Phase	**Uniform (s)**	**Skewed (s)**	**Delta**
Net Comp	0.056	0.160	+186%
Comm	0.535	0.626	+17%
Join	0.085	1.103	+1198%
Total MPI	0.647	1.764	+172%
Speedup	6.26×	1.97×	—

Table 6: Phase breakdown: uniform vs. skewed dataset (4 nodes, 32 tasks, $N = 50$M).

The dominant bottleneck is computational imbalance, not network contention. When 50% of keys share a single value, the macro-partitioning hash routes all matching records to one rank. That rank receives approximately 25M records from each relation, while the remaining 31 ranks share the other 50M. The join imbalance metric (1.039 s out of 1.103 s total join time) confirms that a single rank accounts for 94% of all join work. Furthermore, all ranks simultaneously route duplicate-key records to the same destination, raising communication time by 17% (0.535 s $\rightarrow$ 0.626 s). However, this effect is secondary. The speedup collapse from 6.26× to 1.97× is driven almost entirely by join imbalance; the network is not the limiting factor here.

Mathematical Validation of $P = 512$ in the MPI Architecture The value $P_{\text{local}} = 512$ is chosen so that each micro-partition fits within the L2 cache during the build–probe loop. After macro-partitioning, each rank holds N/R records per relation. The per-partition size for a single relation is:

$$\text{Partition Size} = \frac{N/R}{P_{\text{local}}} \times 8 \text{ B}$$

- **8T/N** ($R = 32$, $N = 50$M): $\dfrac{50\text{M}/32}{512} \approx 3{,}052$ rec $\times$ 8 B ≈ 24.4 KB — fits in L2.
- **16T/N** ($R = 64$, $N = 50$M): $\dfrac{50\text{M}/64}{512} \approx 1{,}526$ rec $\times$ 8 B ≈ 12.2 KB — fits in L1/L2.